

REFLEX - Final Deck Integration

The Problem

Key Take-away: WhatsApp and phone calls are not a reliable delivery coordination system.

Small retailers currently coordinate deliveries through scattered WhatsApp messages and phone calls. This makes it difficult to know which deliveries are open, who is responsible for each delivery, what stage an order has reached, and whether the customer actually received the item.

Reflex replaces fragmented communication with a single, traceable delivery workflow.

Reflex

Key Take-away: Reflex creates visibility and accountability from request creation to delivery completion.

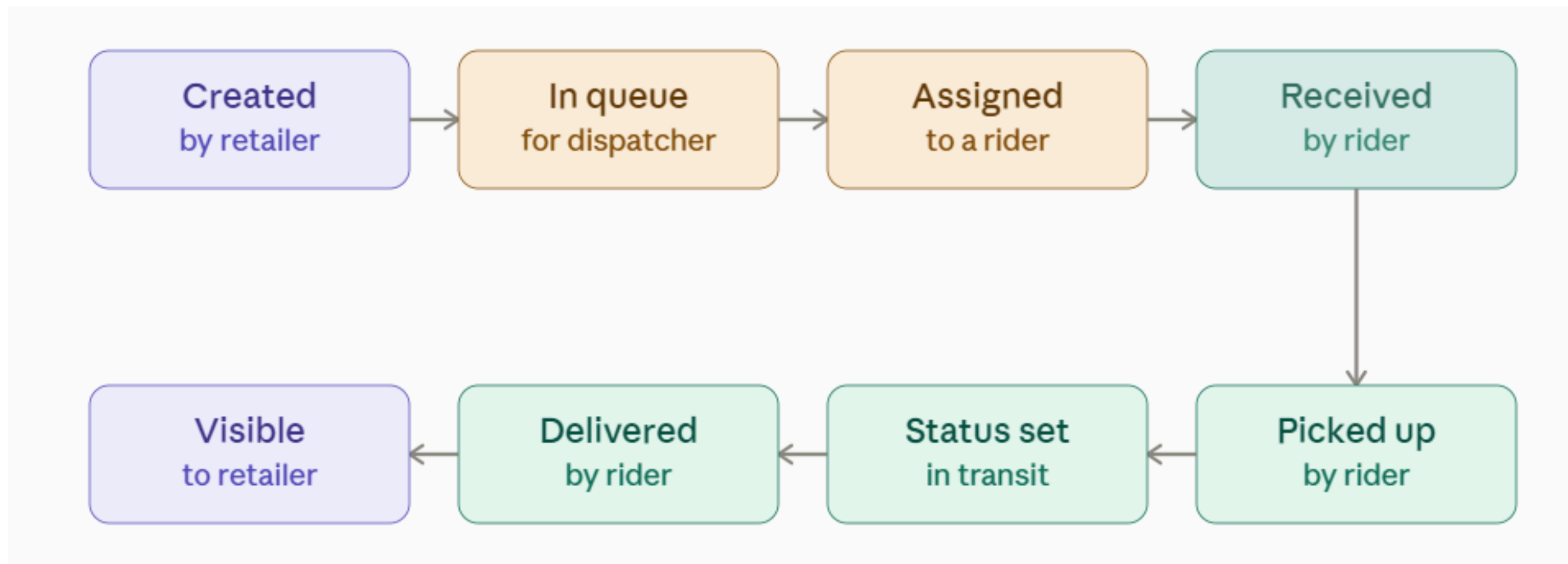
The retailer creates a request containing the customer name, phone number, delivery address and item description. The dispatcher assigns the request to an available rider. The rider then updates the delivery status as the order progresses.

Core status flow: Requested → Assigned → Picked Up → In Transit → Delivered

This creates a reliable record of responsibility and delivery progress.

How It Works

Key Take-away: Every delivery has a clear owner and a visible status.



The workflow reduces ambiguity by ensuring that every important transition is recorded.

Architecture

Key Take-away:

The architecture separates user interactions from delivery coordination logic and data.

Reflex — Architecture

Stack

Layer	Choice	Why
Frontend	React	Single codebase, works in any phone browser, no install
Backend	Node.js + Express	Fast to build, JSON-native
Database	PostgreSQL	Relational integrity between users/requests
Real-time	Socket.io	Live updates for dispatcher/rider views
Scanning	QR (html5-qrcode)	No native app needed
Outside app	SMS (Africa's Talking)	Reaches customers without the app



The architecture separates user interactions from delivery coordination logic and data.

The design prioritizes simplicity because the target users are small retailers who need a practical operational tool rather than a complex enterprise platform.

Data + Status Flow

Key Take-away:

Reflex separates the user interface, delivery logic and data storage to create a simple and reliable coordination system.

Data Model

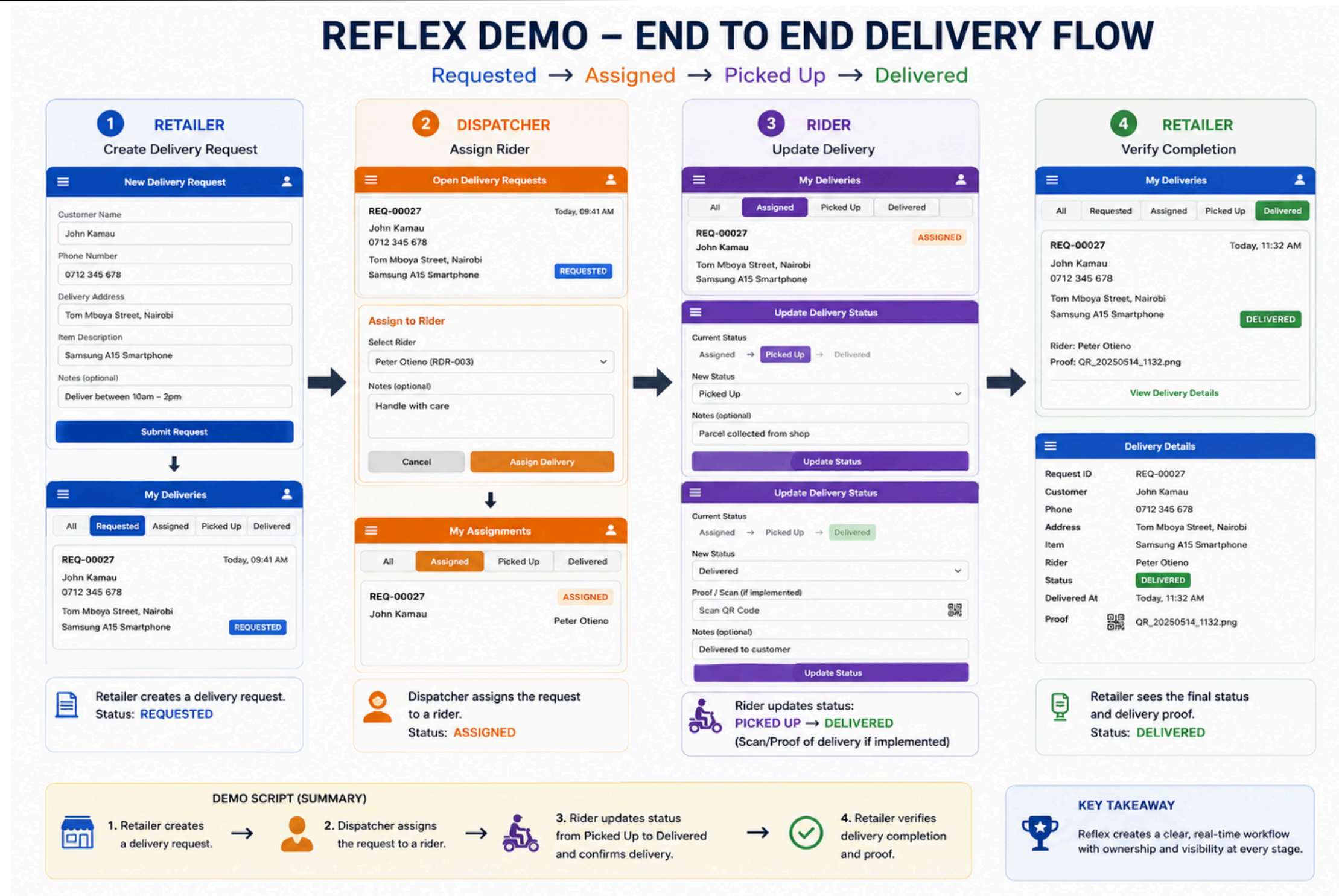
User → Delivery Request → Assignment → Status History → Proof of Delivery

Status Flow

Request Created → Dispatcher Assigns Rider → Rider Picks Up → Rider Delivers → Status Updated

Demo

Key Take-away: The demo proves that Reflex works as one connected workflow from delivery request to completion.



Trade-offs

Key Take-away:

We deliberately chose a focused MVP instead of attempting to solve every delivery problem at once.

Internet dependency

The sprint prioritised proving the core delivery workflow: request → assignment → pickup → delivery. Building complete offline synchronisation would add significant complexity.

Simplified authentication and role management

The prototype prioritised demonstrating the core delivery workflow and user roles rather than implementing full production-level security.

Limited scalability

The current goal is to demonstrate and defend the MVP, not to build production infrastructure for very large numbers of users and deliveries.

Edge Cases

Key Take-away: The team understands what can fail and has defined how the system should respond.

Duplicate scan	Prevents the same order from being confirmed twice.
-----------------------	---

Failed status update	Keeps the previous status and allows the update to be retried.
-----------------------------	--

Poor connectivity	Prevents data loss and synchronizes updates when the connection is restored, where offline support is implemented.
--------------------------	--

Double assignment	Prevents the same delivery from being assigned to multiple riders.
--------------------------	--

Invalid request details	Validates required customer, phone, address and item information before submission.
--------------------------------	---

Incorrect status transition	Only allows valid progression: Requested → Assigned → Picked Up → Delivered.
------------------------------------	--

Already delivered order	Prevents further changes after completion unless authorized.
--------------------------------	--

Roadmap

Key Take-away: The prototype provides a practical foundation for realistic future improvements.

Phase 1 — Current MVP

- Delivery request creation
- Dispatcher-to-rider assignment
- Delivery status tracking
- Role-based workflows
- Centralized delivery records

Phase 2 — Reliability

- Real-time synchronization
- Offline updates with automatic synchronization
- Improved input validation
- Notifications for important status changes
- Stronger audit/history tracking

Phase 3 — Delivery Verification

- QR/barcode scanning
- Digital proof of delivery
- Order confirmation
- Timestamped delivery evidence

Phase 4— Operational Improvement

- Delivery performance dashboard
- Rider performance tracking
- Delivery analytics
- Route/location support

Team Handoffs

Key Take-away: Reflex replaces fragmented delivery communication with a visible, trackable workflow

Reflex turns fragmented delivery communication into one visible, trackable workflow.

It helps teams stay aligned, reduce missed handoffs, improve accountability, and keep deliveries moving efficiently.

The Value

Less confusion → Better coordination → Faster, more reliable delivery